

Técnicas Digitales 2 - Trabajo práctico N° 2 - Act N°4

Braida Agustín (404621) - Ernst Pedro (400624) - Polizze Tomas (400793) - Valentino Rao (402308)

**Técnicas Digitales 2 4R2, Ingeniería Electrónica, Facultad Regional Córdoba
Universidad Tecnológica Nacional**

I. RESUMEN

En esta actividad se implementa un sistema de comunicación bidireccional entre una PC que corre una terminal interactiva bajo sistema operativo Linux y una placa de desarrollo BluePill basada en el microcontrolador STM32F103 (Cortex-M3). Se utiliza el estándar industrial CMSIS para configurar el hardware. A diferencia de las actividades previas que utilizaban lectura por sondeo (polling), en esta ocasión se incorpora la recepción por interrupción (USART1 RXNEIE), mejorando drásticamente la eficiencia en el uso de CPU del microcontrolador al reaccionar de manera asíncrona ante la llegada de caracteres. Se expone un menú interactivo en la PC para encender, apagar y hacer destellar el LED integrado en la placa (PC13), con su respectivo acuse de recibo y retroalimentación bidireccional.

II. INTRODUCCIÓN

La comunicación UART (Universal Asynchronous Receiver-Transmitter) es uno de los métodos de transmisión de datos en serie más difundidos en la ingeniería de sistemas embebidos. En las actividades iniciales de este trabajo práctico, el intercambio de datos se realizaba mediante sondeo continuo de banderas de hardware. Sin embargo, este paradigma consume excesivo tiempo de procesador en bucles vacíos y disminuye la capacidad del sistema para responder a otros estímulos en tiempo real.

El presente informe detalla el diseño y desarrollo de la Actividad 4, donde se migra la recepción a un esquema de interrupción por hardware de comunicaciones. Se describe la configuración a nivel de registros CMSIS y se presenta la estructura de la aplicación de control del lado de la PC.

III. IMPLEMENTACIÓN DE LA COMUNICACIÓN BIDIRECCIONAL

El sistema propuesto consta de dos partes acopladas: el firmware de la BluePill (STM32F103) y la aplicación host interactiva escrita en C para PC bajo sistema operativo Linux. La comunicación se realiza a través de una interfaz UART asíncrona a 9600 bps.

III-A. Esquema de Conexión Física (PC - BluePill)

Para establecer el enlace físico, se utiliza un conversor USB-to-UART (por ejemplo, basado en el chip CP2102 o FT232RL). La conexión eléctrica es cruzada entre las señales de transmisión y recepción, además de requerir una referencia de tierra común indispensable para evitar ruidos y diferencias de potencial:

1. **PA9 (TX de USART1)** de la BluePill se conecta al pin **RXD (Recepción)** del conversor USB-UART.
2. **PA10 (RX de USART1)** de la BluePill se conecta al pin **TXD (Transmisión)** del conversor USB-UART.
3. **GND** de la BluePill se conecta al pin **GND** del conversor USB-UART.

En la Tabla I se detalla el esquema de conexión utilizado.

Tabla I. Conexiones físicas de la interfaz serie

BluePill (STM32F103)	↔	Conversor USB-UART
PA9 (TX)	→	RXD
PA10 (RX)	←	TXD
GND	↔	GND

III-B. Configuración del Periférico USART1

La configuración de USART1 se realizó mediante manipulación directa de registros de hardware. Las especificaciones son: baudrate de 9600 bps, formato de trama 8N1 (8 bits de datos, sin paridad, 1 bit de parada), y habilitación de interrupción por recepción.

■ Configuración de Pines (GPIO):

- Pin PA9 (TX): Se configura como salida de función alternativa push-pull a una velocidad de 2 MHz (bits CNF9[1:0] = 10 y MODE9[1:0] = 10 del registro GPIOA->CRH).
- Pin PA10 (RX): Se configura como entrada flotante (bits CNF10[1:0] = 01 y MODE10[1:0] = 00 del registro GPIOA->CRH).

- **Cálculo de Baudrate (9600 bps):** El cálculo del registro divisor de baudios (USART_BRR) se rige por la fórmula:

$$\text{Baudrate} = \frac{f_{CK}}{16 \times \text{USARTDIV}}$$

Donde f_{CK} es la frecuencia de reloj del periférico USART1, el cual está conectado al bus APB2 que funciona a 72 MHz. Despejando el divisor:

$$USARTDIV = \frac{72 \times 10^6}{16 \times 9600} = 468,75$$

El registro USART1->BRR almacena este valor en formato de coma fija de la siguiente manera:

- Parte entera (Mantisa): 468 en decimal, equivalente a $0 \times 1D4$.
- Parte fraccionaria: $0,75 \times 16 = 12$ en decimal, equivalente a $0 \times C$.

Uniendo ambas partes, se configura USART1->BRR = $0 \times 1D4C$.

■ Formato de Trama (8N1):

- 8 bits de datos: Se limpia el bit M (Word length) en USART1->CR1 (0 por defecto).
- Sin paridad: Se limpia el bit PCE (Parity Control Enable) en USART1->CR1 (0 por defecto).
- 1 bit de parada: Se limpian los bits STOP[1:0] en USART1->CR2 (00 por defecto).

■ Habilitación y Control de Transmisión/Recepción: Se escriben en 1 los bits UE (USART Enable), TE (Transmitter Enable) y RE (Receiver Enable) en USART1->CR1 para poner en funcionamiento el transceptor del periférico.

III-C. Configuración de Interrupciones en el Microcontrolador

Para lograr que la recepción sea asíncrona mediante interrupciones en lugar de polling, se realizaron los siguientes pasos en el firmware:

- Habilitar la máscara de interrupción por receptor no vacío (*Read Data Register Not Empty*): USART1->CR1 |= USART_CR1_RXNEIE. Esto instruye al hardware para generar una petición de interrupción en el NVIC cada vez que se reciba un byte completo.
- Habilitar e indicar la prioridad de la interrupción en el controlador de vectores de interrupción del Cortex-M3 (NVIC) a través de las funciones CMSIS: **Habilitación en el NVIC:**

```
1 NVIC_SetPriority(USART1_IRQn, 2);
2 NVIC_EnableIRQ(USART1_IRQn);
3
```

- Definir la Rutina de Servicio de Interrupción (ISR) USART1_IRQHandler(void) para que recupere el carácter recibido leyendo el registro USART1->DR (acción que automáticamente limpia la bandera de hardware de interrupción RXNE) y levante la bandera global volátil new_command.

III-D. Configuración del Puerto Serie desde la PC (termios)

En el software desarrollado en C para Linux, se configura el puerto serie mediante la API de control de terminal POSIX (termios). El descriptor de archivo aso-

ciado al dispositivo serie se abre con la llamada al sistema `open("/dev/ttyUSB0", O_RDWR | O_NOCTTY)`. A continuación, se detalla la lógica de configuración:

- **Modo Raw (Sin Procesar):** Se aplica la función `cfmakeraw(&tty)`, la cual desactiva el eco local (ECHO), la entrada canónica por líneas (ICANON), y cualquier interpretación de caracteres especiales (como saltos de línea o señales de interrupción). Esto garantiza que la transmisión sea pura a nivel de bytes.
- **Parámetros de Comunicación (9600 8N1):** Se fija la velocidad con `cfsetispeed` y `cfsetospeed` a B9600. Se configura el tamaño de palabra con CS8, se desactiva el bit de paridad (PARENB) y se configura 1 bit de parada (limpiando CSTOPB).
- **Lectura Bloqueante por Carácter:** Se configuran los parámetros de lectura mínima y temporización:

```
1 tty.c_cc[VMIN] = 1;
2 tty.c_cc[VTIME] = 0;
3
```

Esto fuerza a la llamada `read` a bloquear la ejecución hasta que se reciba como mínimo 1 byte desde la BluePill, eliminando la necesidad de realizar sondeo continuo de la UART del lado de la PC.

IV. ESTRUCTURA DEL CÓDIGO Y LÓGICA DE FLUJO

Para garantizar un comportamiento asíncrono y de bajo consumo en el microcontrolador, el sistema se divide en un hilo de ejecución principal y rutinas de atención de interrupciones de hardware.

IV-A. Lógica del Microcontrolador (STM32)

La ejecución de la placa BluePill se divide en dos entornos concurrentes:

- **Programa Principal (Bucle Principal):** Inicializa los periféricos y entra en un bucle `while(1)` de consulta pasiva de la bandera de comando. Al procesar una opción recibida, realiza la acción correspondiente sobre el LED (PC13) y transmite la cadena de acuse de recibo correspondiente.
- **Hilo de Interrupción (USART1 RX):** Se activa de forma asíncrona ante la recepción de un carácter en la línea RX (PA10). Pausa temporalmente el bucle principal, almacena el dato recibido en una variable temporal y activa la bandera que notifica al bucle principal.

IV-B. Variables y Banderas Clave (BluePill)

- **PC13:** Pin asociado al LED incorporado de la placa BluePill (activo en bajo).
- **PA9 / PA10:** Pines asignados a la transmisión (TX) y recepción (RX) de USART1 respectivamente.
- **new_command:** Bandera booleana de software volátil declarada globalmente que indica la presencia de un carácter recibido sin procesar en el bucle principal.
- **command_received:** Registro temporal volátil de 8 bits donde se copia el contenido del registro de datos de recepción del periférico (USART1->DR) durante la rutina de interrupción.

IV-C. Lógica de la Consola Interactiva (PC)

El script en C para la PC opera de manera síncrona: presenta el menú en terminal, lee la opción del usuario por teclado, realiza una limpieza del buffer de entrada serie (`tcflush`), transmite la opción por UART e ingresa a una lectura bloqueante hasta recibir la respuesta de la BluePill terminada en salto de línea.

V. CÓDIGO FUENTE

A continuación, se listan los códigos fuente implementados para ambos extremos de la comunicación. Obsérvese que se han retirado caracteres con acentos de los comentarios del código para garantizar la máxima compatibilidad de compilación de este documento en sistemas LaTeX estándar.

Firmware de la placa BluePill (`src/main.c`):

```
1 #include "stm32f1xx.h"
2 #include <stdint.h>
3
4 typedef enum {
5     LED_STATE_OFF = 0,
6     LED_STATE_ON,
7     LED_STATE_BLINK
8 } led_state_t;
9
10 volatile led_state_t led_mode = LED_STATE_OFF;
11 volatile uint32_t tick_led = 0;
12 volatile char command_received = 0;
13 volatile uint8_t new_command = 0;
14
15 void UART_send_char(char c) {
16     while (!(USART1->SR & USART_SR_TXE));
17     USART1->DR = c;
18 }
19
20 void UART_send_string(const char *str) {
21     while (*str) {
22         UART_send_char(*str);
23         str++;
24     }
25 }
26
27 void SysTick_Handler(void) {
28     if (led_mode == LED_STATE_BLINK) {
29         tick_led++;
30         if (tick_led >= 500) {
31             GPIOC->ODR ^= (1 << 13);
32             tick_led = 0;
33         }
34     }
35 }
36
37 void USART1_IRQHandler(void) {
38     if (USART1->SR & USART_SR_RXNE) {
39         command_received = (char) (USART1->DR &
40                                     0xFF);
41         new_command = 1;
42     }
43 }
44
45 int main(void) {
46     // Habilitar Relojes
47     RCC->APB2ENR |= RCC_APB2ENR_IOPCEN |
48                     RCC_APB2ENR_IOPAEN |
49                     RCC_APB2ENR_USART1EN |
50                     RCC_APB2ENR_AFIOEN;
```

```
48
49 // Configurar SysTick a 1ms
50 SysTick->CTRL &= ~
51 SysTick_CTRL_CLKSOURCE_Msk;
52 SysTick->LOAD = 999;
53 SysTick->VAL = 0;
54 SysTick->CTRL |= SysTick_CTRL_TICKINT_Msk
55 | SysTick_CTRL_ENABLE_Msk;
56
57 // Configurar PC13 como salida push-pull 2
58 MHz
59 GPIOC->CRH &= ~GPIO_CRH_CNF13_Msk;
60 GPIOC->CRH |= GPIO_CRH_MODE13_1;
61 GPIOC->ODR |= (1 << 13);
62
63 // PA9 (TX) - Alternate function push-pull
64 GPIOA->CRH &= ~(0xF << 4);
65 GPIOA->CRH |= (0xB << 4);
66
67 // PA10 (RX) - Input floating
68 GPIOA->CRH &= ~(0xF << 8);
69 GPIOA->CRH |= (0x4 << 8);
70
71 // Configurar USART1
72 USART1->CR1 &= ~USART_CR1_M;
73 USART1->CR1 &= ~USART_CR1_PCE;
74 USART1->CR2 &= ~USART_CR2_STOP;
75 USART1->BRR = 0x1D4C;
76
77 USART1->CR1 |= USART_CR1_UE | USART_CR1_TE
78 | USART_CR1_RE;
79 USART1->CR1 |= USART_CR1_RXNEIE;
80
81 // Configurar NVIC
82 NVIC_SetPriority(USART1_IRQn, 2);
83 NVIC_EnableIRQ(USART1_IRQn);
84
85 while (1) {
86     if (new_command) {
87         new_command = 0;
88         char cmd = command_received;
89
90         if (cmd == '1') {
91             led_mode = LED_STATE_ON;
92             GPIOC->ODR &= ~(1 << 13);
93             UART_send_string("LED
94 encendido\n");
95         }
96         else if (cmd == '2') {
97             led_mode = LED_STATE_OFF;
98             GPIOC->ODR |= (1 << 13);
99             UART_send_string("LED apagado\
100 n");
101         }
102         else if (cmd == '3') {
103             led_mode = LED_STATE_BLINK;
104             UART_send_string("LED
105 parpadeando\n");
106         }
107         else if (cmd == '0') {
108             led_mode = LED_STATE_OFF;
109             GPIOC->ODR |= (1 << 13);
110             UART_send_string("Saliendo\n");
111         }
112     }
113 }
```

Aplicación interactiva de la PC (lecturadesdePC.c):

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <fcntl.h>
4  #include <unistd.h>
5  #include <termios.h>
6  #include <signal.h>
7  #include <string.h>
8
9  #define SERIAL_PORT "/dev/ttyUSB0"
10
11 int fd;
12 struct termios oldtty;
13
14 void cerrar_programa(int sig) {
15     tcsetattr(fd, TCSANOW, &oldtty);
16     close(fd);
17     printf("\nPuerto cerrado.\n");
18     exit(0);
19 }
20
21 int main(void) {
22     struct termios tty;
23
24     signal(SIGINT, cerrar_programa);
25
26     fd = open(SERIAL_PORT, O_RDWR | O_NOCTTY);
27
28     if (fd < 0) {
29         perror("No se pudo abrir el puerto
30 serie /dev/ttyUSB0");
31         return 1;
32     }
33
34     tcgetattr(fd, &oldtty);
35     tty = oldtty;
36
37     cfmakeraw(&tty);
38
39     cfsetispeed(&tty, B9600);
40     cfsetospeed(&tty, B9600);
41
42     tty.c_cflag |= CLOCAL | CREAD;
43     tty.c_cflag &= ~PARENB;
44     tty.c_cflag &= ~CSTOPB;
45     tty.c_cflag &= ~CSIZE;
46     tty.c_cflag |= CS8;
47
48     tty.c_lflag &= ~ECHO;
49
50     tty.c_cc[VMIN] = 1;
51     tty.c_cc[VTIME] = 0;
52
53     tcflush(fd, TCIFLUSH);
54     tcsetattr(fd, TCSANOW, &tty);
55
56     printf("Puerto %s configurado
57 correctamente a 9600 8N1.\n", SERIAL_PORT)
58 ;
59     printf("Ctrl+C para salir de la aplicacion
60 .\n");
61
62     while (1) {
63         printf("\n===== MENU UART =====\n");
64         printf("1: LED_ON\n");
65         printf("2: LED_OFF\n");
66         printf("3: LED_TITILA\n");
67         printf("0: Salir\n");
68         printf("Opcion: ");

```

```

65     fflush(stdout);
66
67     char opcion;
68     if (scanf(" %c", &opcion) != 1) {
69         continue;
70     }
71
72     if (opcion == '0') {
73         write(fd, &opcion, 1);
74         printf("Saliendo de la aplicacion
75 ....\n");
76         break;
77     }
78
79     if (opcion < '1' || opcion > '3') {
80         printf("Opcion no valida. Intente
81 nuevamente.\n");
82         continue;
83     }
84
85     tcflush(fd, TCIFLUSH);
86
87     if (write(fd, &opcion, 1) != 1) {
88         perror("Error de escritura en el
89 puerto serie");
90         break;
91     }
92
93     char respuesta[256];
94     int idx = 0;
95     char c;
96     int n;
97
98     while (idx < 255) {
99         n = read(fd, &c, 1);
100         if (n > 0) {
101             if (c == '\n') {
102                 respuesta[idx] = '\0';
103                 break;
104             }
105             if (c != '\r') {
106                 respuesta[idx++] = c;
107             }
108         } else if (n < 0) {
109             perror("Error de lectura del
110 puerto serie");
111             break;
112         }
113     }
114     printf("BluePill responde: %s\n",
115 respuesta);
116 }

```

VI. PRUEBAS Y VERIFICACIÓN DEL FUNCIONAMIENTO

Para corroborar el correcto desempeño de la comunicación bidireccional y las interrupciones por recepción, se ejecutaron las siguientes pruebas metodológicas:

- **Prueba de Transmisión del Host a la BluePill:** Al seleccionar una opción en el menú en la PC (por ejemplo, '1'), se comprobó el envío correcto de un único byte. Se verificó con un analizador lógico que la trama cumple

con la temporización a 9600 bps y que no existe ruido en la línea.

- **Verificación de la Interrupción por Recepción:** Se comprobó mediante depuración en hardware que al ingresar un carácter, la ejecución del bucle principal de la BluePill se detiene inmediatamente y entra en la ISR `USART1_IRQHandler(void)`, donde el registro `DR` es leído, la bandera `new_command` cambia a 1 y el procesador retoma de inmediato la tarea en segundo plano.
- **Verificación de Respuestas y Control del LED:**
 - **Comando '1':** El LED integrado en la placa BluePill (PC13) se enciende (el pin se establece a nivel bajo). La terminal de la PC muestra por pantalla: `BluePill responde: LED encendido`.
 - **Comando '2':** El LED se apaga (pin en alto). La PC muestra: `BluePill responde: LED apagado`.
 - **Comando '3':** El LED parpadea periódicamente cada 500 ms de forma asíncrona mediante el conteo del `SysTick`. Se corroboró que el parpadeo del LED no bloquea la terminal serie de la PC, pudiendo enviar nuevos comandos en paralelo. La PC muestra: `BluePill responde: LED parpadeando`.
 - **Comando '0':** La BluePill detiene el parpadeo y apaga el LED. Envía a la PC la cadena `Saliendo` y la aplicación del host cierra de manera ordenada el descriptor de archivo de la UART.

VII. CONCLUSIÓN

La realización de este trabajo práctico permitió consolidar la implementación de la comunicación serie asíncrona mediante manipulación directa de registros de hardware y la configuración de interrupciones asíncronas en arquitecturas Cortex-M3.

- **Dificultades Encontradas:** La principal dificultad consistió en la configuración inicial de la librería `termios` en Linux. Lograr que el sistema operativo entregara directamente cada byte sin procesar y de manera no bloqueante requirió una minuciosa lectura de las opciones de control de la terminal POSIX.
- **Errores Frecuentes:** Durante las pruebas iniciales, el error más común fue olvidar realizar una conexión de referencia común de masa (GND) entre el conversor USB-UART y la BluePill. Esto generó errores de trama y recepción de caracteres erróneos. Otro fallo habitual fue no limpiar la bandera de interrupción de hardware, lo cual se solucionó al comprender que la lectura del registro `DR` realiza esta tarea de manera automática.
- **Conceptos más Significativos:** Se comprendió la diferencia crítica en términos de eficiencia de CPU entre la lectura por sondeo (polling) y el esquema basado en interrupciones (ISR). Las interrupciones permiten al microcontrolador entrar en modos de bajo consumo o ejecutar otras tareas secuenciales de fondo sin perder eventos externos de comunicación.
- **Aprendizajes Obtenidos:** Se consolidó el uso del calificador `volatile` en variables globales compartidas entre el programa principal y las ISRs para evitar optimizaciones del compilador que oculten los cambios de estado.

Además, se asimiló la lógica de diseño de sistemas de comunicación interactivos Host-Target.

REFERENCIAS

- [1] STMicroelectronics, *STM32F103xx Reference Manual (RM0008)*, Rev. 21, 2021.
- [2] ARM Limited, *Cortex Microcontroller Software Interface Standard (CMSIS) documentation*, 2023.